

1914 项目初始化 + 单卡原型 — 实现计划

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: 搭建 Godot 4.x 项目骨架，实现第一张可拖拽卡牌放到棋盘格子上

Architecture: Godot 4.x 原生 2D 项目，GDScript 脚本。按技术文档规划的目录结构搭建，Autoload 单例 **GameManager** 管理全局状态，**BattleScene** 承载棋盘+卡牌交互

Tech Stack: Godot 4.x, GDScript, JSON

Global Constraints

- Godot 4.x (建议 4.3+)，从 <https://godotengine.org/download> 下载标准版即可
- 纯 2D 项目，不使用 3D 节点
- 卡牌数据用 JSON 文件存储，非程序员可编辑
- 项目目录结构遵循 [docs/superpowers/specs/2026-08-05-ww1-card-game-design.md](#) 第 6 节

前置条件

- ☐ **下载安装 Godot 4.x:** 访问 <https://godotengine.org/download>，下载 Windows 版 Godot 4.x (Standard version)，解压到本地即可运行，无需安装

Task 1: 初始化 Godot 项目与目录结构

Files:

- Create: **project.godot**
- Create: **data/cards/units/.gitkeep**

- Create: `data/cards/orders/.gitkeep`
- Create: `data/nations.json`
- Create: `assets/card_art/.gitkeep`
- Create: `assets/icons/.gitkeep`
- Create: `assets/fonts/.gitkeep`
- Create: `assets/sounds/.gitkeep`
- Create: `scenes/main_menu.tscn`
- Create: `scenes/battle.tscn`
- Create: `scenes/deck_editor.tscn`
- Create: `scenes/ui_components/.gitkeep`
- Create: `scripts/core/game_logic.gd`
- Create: `scripts/core/card_data.gd`
- Create: `scripts/core/turn_manager.gd`
- Create: `scripts/core/battle_state.gd`
- Create: `scripts/network/network_manager.gd`
- Create: `scripts/ui/card_display.gd`
- Create: `scripts/ui/hand_manager.gd`
- Create: `scripts/ui/board.gd`
- Create: `scripts/autoload/game_manager.gd`
- Create: `themes/ww1_theme.tres`

Interfaces:

- Consumes: nothing
- Produces: project root with all directories ready, empty `.gd` stub scripts, empty `.tscn` placeholder scenes

- ☐ **Step 1: 用 Godot 编辑器创建新项目**

打开 Godot 4.x, 点击"新建项目", 项目名称填 `1914`, 项目路径选 `D:\Code\1914`, 渲染器选 `Forward+` (默认), 点击"创建" → 这会自动生成 `project.godot`

- ☐ **Step 2: 在 Godot 编辑器中创建目录结构**

使用 Godot 编辑器底部的 FileSystem 面板, 右键逐一创建以下目录:

```
data/cards/units/
data/cards/orders/
assets/card_art/
assets/icons/
assets/fonts/
assets/sounds/
```

```
scenes/ui_components/  
scripts/core/  
scripts/network/  
scripts/ui/  
scripts/autoload/  
themes/
```

- ❑ **Step 3: 创建 nations.json 占位数据**

在 FileSystem 面板右键 **data/** → New Resource，或直接在项目中新建文本文件：

```
{  
  "nations": []  
}
```

- ❑ **Step 4: 创建 Autoload 单例 — GameManager**

1. 在 **scripts/autoload/** 目录右键 → 新建脚本，命名为 **game_manager.gd**
2. 脚本内容：

```
extends Node  
  
## 全局游戏状态管理  
## 当前游戏模式：local（本地对战）/ online（在线对战）  
var game_mode: String = "local"  
  
func _ready() -> void:  
    print("[GameManager] Initialized")
```

3. 打开菜单 Project → Project Settings → Autoload，点击 Path 旁的文件夹图标，选择 **scripts/autoload/game_manager.gd**，Node Name 保持 **GameManager**，点击 Add

- ❑ **Step 5: 创建 BattleScene 基础骨架**

1. 在 **scenes/** 目录下新建场景 → 选 **Node2D** 作为根节点，命名为 **BattleScene**，保存为 **battle.tscn**
2. 为根节点附加脚本 **scripts/ui/board.gd**：

```
extends Node2D  
  
## Board 根节点 - 管理棋盘所有格子和单位
```

```
func _ready() ->void:
    print("[Board] BattleScene loaded")
```

- ☐ **Step 6: 设置默认启动场景**

Project Settings → General → Application → Run → Main Scene, 设为 `scenes/battle.tscn`

- ☐ **Step 7: 验证 — 运行项目**

按 F5 或点击右上角 Run Project 按钮。预期：窗口启动，控制台输出 `[GameManager] Initialized` 和 `[Board] BattleScene loaded`

- ☐ **Step 8: Commit**

```
git init
git add -A
git commit -m "feat: initialize Godot 4.x project with directory structure and autoload"
```

Task 2: 定义卡牌数据结构（CardData Resource）

Files:

- Create: `scripts/core/card_data.gd`
- Create: `data/cards/units/test_infantry.json`

Interfaces:

- Consumes: project structure from Task 1
- Produces: `CardData` Resource 类 + `CardDataLoader` 单例，供后续任务使用
 - `CardData` — `Resource` 子类，字段： `id: String`, `card_name: String`, `nation: String`, `type: String`, `unit_class: String`, `cost_g: int`, `cost_k: int`, `attack: int`, `defense: int`, `vision_range: String`, `attack_range: String`, `abilities: Array[String]`, `rarity: String`, `art: String`, `flavor_text: String`

- **CardDataLoader** — Autoload 单例，方法 `load_all_cards()` -> **Dictionary** (返回 `{id: CardData}`)

- ☐ **Step 1: 创建 CardData Resource 类**

在 `scripts/core/card_data.gd` 中：

```
extends Resource
class_name CardData

## 卡牌数据结构，对应 JSON 中一张卡牌的所有字段

@export var id: String = ""
@export var card_name: String = ""
@export var nation: String = ""
@export var type: String = ""          # "unit" 或 "order"
@export var unit_class: String = ""    #
cavalry/infantry/tank/fighter/bomber/artillery/fortification
@export var cost_g: int = 0            # 生产所需经济
@export var cost_k: int = 0            # 部署所需指挥点
@export var attack: int = 0
@export var defense: int = 0
@export var vision_range: String = ""  # adjacent_4 / adjacent_8_forward /
front_3x2 等
@export var attack_range: String = ""
@export var abilities: Array[String] = []
@export var rarity: String = "common"  # common / silver / gold
@export var art: String = ""
@export var flavor_text: String = ""
```

- ☐ **Step 2: 创建 CardDataLoader Autoload 单例**

在 `scripts/core/card_data.gd` 末尾追加（同一文件）：

```
class_name CardDataLoader
extends Node

## 游戏启动时扫描 data/cards/ 目录，加载所有卡牌 JSON

var cards: Dictionary = {} # {id: CardData}

func load_all_cards() -> Dictionary:
    cards.clear()
    _load_from_dir("res://data/cards/units/")
    _load_from_dir("res://data/cards/orders/")
    print("[CardDataLoader] Loaded %d cards" % cards.size())
    return cards

func _load_from_dir(dir_path: String) -> void:
    var dir = DirAccess.open(dir_path)
    if dir == null:
        printerr("[CardDataLoader] Cannot open directory: %s" % dir_path)
```

```

        return
    dir.list_dir_begin()
    var file_name = dir.get_next()
    while file_name != "":
        if file_name.ends_with(".json"):
            _load_card(dir_path + file_name)
        file_name = dir.get_next()
    dir.list_dir_end()

func _load_card(file_path: String) -> void:
    var file = FileAccess.open(file_path, FileAccess.READ)
    if file == null:
        printerr("[CardDataLoader] Cannot read file: %s" % file_path)
        return
    var json_text = file.get_as_text()
    file.close()
    var json = JSON.new()
    var error = json.parse(json_text)
    if error != OK:
        printerr("[CardDataLoader] JSON parse error in %s" % file_path)
        return
    var data = json.get_data()
    var card = CardData.new()
    card.id = data.get("id", "")
    card.card_name = data.get("name", "")
    card.nation = data.get("nation", "")
    card.type = data.get("type", "")
    card.unit_class = data.get("unit_class", "")
    card.cost_g = data.get("cost_g", 0)
    card.cost_k = data.get("cost_k", 0)
    card.attack = data.get("attack", 0)
    card.defense = data.get("defense", 0)
    card.vision_range = data.get("vision_range", "")
    card.attack_range = data.get("attack_range", "")
    card.abilities.assign(data.get("abilities", []))
    card.rarity = data.get("rarity", "common")
    card.art = data.get("art", "")
    card.flavor_text = data.get("flavor_text", "")
    cards[card.id] = card

```

- ❑ **Step 3: 注册 CardDataLoader 为 Autoload**

Project → Project Settings → Autoload, 添加 `scripts/core/card_data.gd`, Node Name 填 `CardDataLoader`

- ❑ **Step 4: 创建测试卡牌 JSON**

新建 `data/cards/units/test_infantry.json`:

```

{
  "id": "test_infantry_01",
  "name": "测试步兵",
  "nation": "neutral",
  "type": "unit",

```

```
"unit_class": "infantry",
"cost_g": 2,
"cost_k": 1,
"attack": 2,
"defense": 3,
"vision_range": "adjacent_4",
"attack_range": "adjacent_4",
"abilities": [],
"rarity": "common",
"art": "",
"flavor_text": "一战的士兵们迈入堑壕。"
}
```

- ☐ **Step 5: 验证 — 运行并检查控制台**

按 F5 运行，预期控制台输出 `[CardDataLoader] Loaded 1 cards`

- ☐ **Step 6: Commit**

```
git add -A
git commit -m "feat: add CardData resource class and CardDataLoader autoload with
test card"
```

Task 3: 实现单卡拖拽原型（Board + Card）

Files:

- Create: `scenes/ui_components/card.tscn`
- Create: `scenes/ui_components/board_slot.tscn`
- Create: `scripts/ui/card_display.gd`
- Modify: `scripts/ui/board.gd`
- Modify: `scenes/battle.tscn`

Interfaces:

- Consumes: `CardData` (Task 2), `CardDataLoader.load_all_cards()` (Task 2)
- Produces:
 - `CardDisplay` — `Node2D` 脚本，负责单张卡牌的渲染和拖拽
 - `BoardSlot` — `Node2D` 脚本，代表棋盘上一个可接受卡牌的格子
 - `Board` 更新为生成 5×5 棋盘格子并支持拖放

- ❑ **Step 1: 创建 BoardSlot 场景与脚本**

`scripts/ui/card_display.gd` 中先放 BoardSlot（同一文件，后续 Task 4 拆分）：

`scripts/ui/board.gd` 中追加 BoardSlot 内部类：

实际做法：在 Godot 中 `scenes/ui_components/` 右键新建脚本 `board_slot.gd`

```
extends Node2D
class_name BoardSlot

## 棋盘上的一个格子，可接受卡牌拖放

@export var slot_row: int = 0
@export var slot_col: int = 0

var occupied_card: Node2D = null

func _ready() -> void:
    pass

func can_accept_card() -> bool:
    return occupied_card == null

func place_card(card: Node2D) -> void:
    occupied_card = card
    card.position = Vector2.ZERO
    add_child(card)
    print("[BoardSlot] Card placed at (%d, %d)" % [slot_row, slot_col])

func remove_card() -> void:
    occupied_card = null
```

- ❑ **Step 2: 创建 Card 场景与脚本**

`scenes/ui_components/` 右键新建场景，根节点 `Node2D`，保存为 `card.tscn`。

`scripts/ui/` 右键新建脚本 `card_display.gd`：

```
extends Node2D
class_name CardDisplay

## 单张卡牌的显示与拖拽行为

@export var card_data: CardData = null

var is_dragging: bool = false
var drag_offset: Vector2 = Vector2.ZERO
var original_position: Vector2 = Vector2.ZERO
var original_parent: Node = null

func setup(data: CardData) -> void:
```

```

card_data = data
# 用 ColorRect 临时表示卡牌外观（后续替换为正式美术）
var bg = ColorRect.new()
bg.size = Vector2(80, 100)
bg.color = Color(0.3, 0.4, 0.5, 1.0)
add_child(bg)

var label = Label.new()
label.text = data.card_name
label.add_theme_font_size_override("font_size", 12)
label.position = Vector2(4, 4)
add_child(label)

print("[CardDisplay] Setup: %s" % data.card_name)

func _input(event: InputEvent) -> void:
    if event is InputEventMouseButton:
        if event.button_index == MOUSE_BUTTON_LEFT:
            if event.pressed:
                _try_start_drag(event.position)
            else:
                _try_end_drag(event.position)
    if is_dragging and event is InputEventMouseMotion:
        global_position = event.position - drag_offset

func _try_start_drag(mouse_pos: Vector2) -> void:
    # 检查鼠标是否在卡牌范围内
    var card_rect = Rect2(global_position, Vector2(80, 100))
    if card_rect.has_point(mouse_pos):
        is_dragging = true
        original_position = global_position
        original_parent = get_parent()
        drag_offset = mouse_pos - global_position
        # 提升到最顶层
        var root = get_tree().current_scene
        var old_global = global_position
        get_parent().remove_child(self)
        root.add_child(self)
        global_position = old_global
        print("[CardDisplay] Drag start: %s" % card_data.card_name)

func _try_end_drag(mouse_pos: Vector2) -> void:
    if not is_dragging:
        return
    is_dragging = false
    # 检测放置目标
    var dropped = false
    var board_slots = get_tree().get_nodes_in_group("board_slot")
    for slot in board_slots:
        var slot_rect = Rect2(slot.global_position, Vector2(80, 100))
        if slot_rect.has_point(mouse_pos) and slot.can_accept_card():
            var root = get_tree().current_scene
            root.remove_child(self)
            slot.place_card(self)
            dropped = true
            break
    if not dropped:
        # 回到原位
        var root = get_tree().current_scene

```

```
root.remove_child(self)
original_parent.add_child(self)
global_position = original_position
print("[CardDisplay] Drag cancelled – returned to original position")
```

- ❑ **Step 3: 更新 Board 脚本 — 生成 5×5 棋盘**

更新 `scripts/ui/board.gd`:

```
extends Node2D

## Board 根节点 – 管理 5×5 棋盘格子和单位

const GRID_SIZE: int = 5
const SLOT_SPACING: Vector2 = Vector2(90, 115)
const SLOT_SIZE: Vector2 = Vector2(80, 100)
const GRID_OFFSET: Vector2 = Vector2(200, 50)

func _ready() -> void:
    print("[Board] BattleScene loaded")
    _create_board()

func _create_board() -> void:
    var slot_scene = load("res://scenes/ui_components/board_slot.tscn")
    for row in range(GRID_SIZE):
        for col in range(GRID_SIZE):
            var slot: BoardSlot = slot_scene.instantiate()
            slot.slot_row = row
            slot.slot_col = col
            slot.position = GRID_OFFSET + Vector2(col * SLOT_SPACING.x, row *
SLOT_SPACING.y)
            slot.add_to_group("board_slot")
            add_child(slot)
    print("[Board] Created %d slots" % (GRID_SIZE * GRID_SIZE))

func spawn_card(card_data: CardData) -> CardDisplay:
    var card_scene = load("res://scenes/ui_components/card.tscn")
    var card: CardDisplay = card_scene.instantiate()
    card.setup(card_data)
    return card
```

- ❑ **Step 4: 更新 BattleScene — 进入时生成一张测试卡牌**

修改 `scripts/ui/board.gd` 的 `_ready()`，在棋盘生成后创建一张卡：

```
func _ready() -> void:
    print("[Board] BattleScene loaded")
    _create_board()
    _spawn_test_card()

func _spawn_test_card() -> void:
```

```
var card_data = CardDataLoader.load_all_cards().get("test_infantry_01")
if card_data == null:
    printerr("[Board] Test card not found!")
    return
var card = spawn_card(card_data)
card.position = Vector2(50, 300)
add_child(card)
print("[Board] Test card spawned at edge")
```

- ☐ **Step 5: 创建 board_slot.tscn 场景**

在 `scenes/ui_components/` 右键新建场景 → Node2D → 保存为 `board_slot.tscn`，附加 `scripts/ui/board_slot.gd`（注意：需要把 board_slot 脚本独立成一个文件）。给 BoardSlot 添加视觉标识：

在 board_slot.tscn 中，为根节点添加子节点 `ColorRect`，size (80, 100)，color 半透明灰色 (`0.2, 0.2, 0.2, 0.3`)。

- ☐ **Step 6: 把 BoardSlot 脚本拆分为独立文件**

把 Task 3 Step 1 中的 BoardSlot 类从 board.gd 中移出，放入独立文件 `scripts/ui/board_slot.gd`。

- ☐ **Step 7: 验证 — 运行并拖拽卡牌**

1. 按 F5 运行
2. 预期看到 5×5 灰色格子网格 + 一张灰色卡牌"测试步兵"
3. 用鼠标拖拽卡牌，拖到格子上松手，卡牌应吸附到该格子
4. 拖到空白处松手，卡牌应回到原位

- ☐ **Step 8: Commit**

```
git add -A
git commit -m "feat: implement single card drag-and-drop onto 5x5 board grid"
```

Task 4: 棋盘尺寸支持动态扩展 (5×5 ↔ 7×5)

Files:

- Modify: `scripts/ui/board.gd`

Interfaces:

- Consumes: `Board._create_board()` from Task 3
- Produces: `Board.add_extra_row(is_top: bool)` — 添加一行到顶部或底部
 - `Board.max_rows: int = 5` — 当前最大行数
- ☐ **Step 1: 添加动态行扩展方法**

```
var max_rows: int = 5

func add_extra_row(is_top: bool) -> void:
    if max_rows >= 7:
        print("[Board] Max rows already 7, cannot add more")
        return
    var new_row = max_rows - 5 + 1 # 当前是第几额外行
    var row_idx: int
    var y_offset: float
    if is_top:
        row_idx = 0 # 最上方
        # 把现有所有行往下移一行
        for slot in get_tree().get_nodes_in_group("board_slot"):
            slot.slot_row += 1
            slot.position.y += SLOT_SPACING.y
    else:
        row_idx = max_rows # 最下方
    # 创建新行
    var slot_scene = load("res://scenes/ui_components/board_slot.tscn")
    for col in range(5):
        var slot: BoardSlot = slot_scene.instantiate()
        slot.slot_row = row_idx
        slot.slot_col = col
        y_offset = GRID_OFFSET.y + row_idx * SLOT_SPACING.y
        slot.position = Vector2(GRID_OFFSET.x + col * SLOT_SPACING.x, y_offset)
        slot.add_to_group("board_slot")
        add_child(slot)
    max_rows += 1
    print("[Board] Added extra row %s, total rows: %d" % ["top" if is_top else "bottom", max_rows])
```

- ☐ **Step 2: 验证**

F5 运行后在 Godot 编辑器的 Remote 场景树中，应该能看到 5 行格子。逻辑上 `add_extra_row(false)` 可在以后由卡牌效果调用。

- ☐ **Step 3: Commit**

```
git add -A
git commit -m "feat: support dynamic board row expansion from 5 to 7"
```

验证清单

全部 Task 完成后，确认以下行为：

1. Godot 编辑器可一键运行（F5），无报错
2. 控制台输出 `[GameManager] Initialized`、`[CardDataLoader] Loaded 1 cards`、`[Board] BattleScene loaded`、`[Board] Created 25 slots`
3. 界面显示 5×5 灰色棋盘网格
4. 棋盘外有一张"测试步兵"卡牌
5. 卡牌可鼠标拖拽到任意空格子并吸附
6. 拖到非格子区域卡牌回到原位
7. `data/cards/units/test_infantry.json` 非程序员可以直接编辑修改数值